

Explain Good Numbers Code

This code is solving **LeetCode 1922 — Count Good Numbers**. The main idea is actually very simple once you understand what makes a number “good.”

1. What is the question asking?

You are given an integer `n`.

You need to count how many **strings/numbers of length `n`** satisfy these rules:

- At **even indices** (`0, 2, 4, ...`), the digit must be an **even digit**:

```
0, 2, 4, 6, 8
```

→ **5 choices**

- At **odd indices** (`1, 3, 5, ...`), the digit must be a **prime digit**:

```
2, 3, 5, 7
```

→ **4 choices**

Return the answer modulo:

```
10^9 + 7
```

Example: `n = 4`

Positions:

index:	0	1	2	3
	↓	↓	↓	↓
	E	P	E	P

Even positions:

```
0,2,4,6,8 → 5 choices
```

Odd positions:

```
2,3,5,7 → 4 choices
```

Therefore:

```
5 × 4 × 5 × 4
= 52 × 42
= 400
```

So:

```
n = 4
answer = 400
```

2. The key observation

The most important part of the problem is realizing that **each position is independent**.

For every even index:

```
5 choices
```

For every odd index:

```
4 choices
```

So instead of generating all possible numbers, we simply count the number of choices.

3. How many even positions are there?

Your code:

```
long long even = (n+1)/2;
```

Why?

Indices start from `0`.

For example, if:

```
n = 5
```

indices are:

```
0 1 2 3 4
E O E O E
```

Even indices:

```
0, 2, 4
```

There are:

```
3
```

Formula:

```
(n + 1) / 2
```

For n = 5:

```
(5 + 1) / 2
= 6 / 2
= 3
```

For n = 4:

```
0 1 2 3
E O E O
```

There are 2 even positions:

```
(4 + 1) / 2
= 5 / 2
= 2
```

Because integer division removes the decimal part.

Therefore:

```
even = (n + 1) / 2;
```

4. How many odd positions?

Your code:

```
long long odd = n/2;
```

Again because indices start at 0.

For:

```
n = 5
```

```
0 1 2 3 4
E O E O E
```

Odd positions:

```
1, 3
```

So:

```
5 / 2 = 2
```

For:

```
n = 4
```

```
0 1 2 3
E O E O
```

There are 2 odd positions:

$$4 / 2 = 2$$

Hence:

$$\text{odd} = n / 2;$$

5. Why 5^{even} ?

Every even position has 5 possible digits:

0
2
4
6
8

Suppose we have 3 even positions:

$_ _ _ _ _$
E O E O E

For the first even position:

5 choices

Second:

5 choices

Third:

5 choices

Using the multiplication principle:

$$5 \times 5 \times 5 \\ = 5^3$$

Therefore:

```
long long evenpart = modpow(5, even);
```

means:

```
number of ways to fill all even positions  
=  
5^number_of_even_positions
```

6. Why 4^{odd} ?

At every odd position, we need a prime digit.

The single-digit primes are:

2, 3, 5, 7

So:

4 choices

If there are 3 odd positions:

$$4 \times 4 \times 4 \\ = 4^3$$

Therefore:

```
long long oddpart = modpow(4, odd);
```

means:

```
number of ways to fill odd positions  
=  
4^number_of_odd_positions
```

7. Why multiply the two?

Now we have:

```
evenpart = 5^even
oddpart  = 4^odd
```

The choices for the two types of positions are independent.

Therefore:

```
Total = 5^even × 4^odd
```

Your code:

```
return (evenpart * oddpart) % MOD;
```

That's the entire mathematical solution.

8. Let's trace `n = 5`

Suppose:

```
n = 5
```

Positions:

```
index:  0  1  2  3  4
        E  O  E  O  E
```

Number of even positions:

```
even = (5 + 1) / 2
      = 3
```

Number of odd positions:

```
odd = 5 / 2
     = 2
```

Therefore:

```
evenpart = 53 = 125
```

```
oddpart  = 42 = 16
```

Total:

```
125 × 16
= 2000
```

So:

```
n = 5
answer = 2000
```

9. The difficult part: Why do we need `modpow()` ?

Now consider:

```
n = 1,000,000,000
```

We would need:

```
5500000000 × 4500000000
```

We obviously cannot calculate that normally.

Even:

```
pow(5, 500000000)
```

would not be suitable because the number is astronomically large.

The problem asks for the answer modulo:

1,000,000,007

So we use modular exponentiation.

10. Your recursive `modpow()`

This function:

```
long long modpow(long long base, long long pow) {
    base %= MOD;

    if(pow == 0)
        return 1;

    if(pow % 2 == 1) {
        return (base * modpow(base, pow-1)) % MOD;
    }
    else {
        return modpow((base * base) % MOD, pow/2) % MOD;
    }
}
```

The basic mathematical idea is:

If exponent is even

Suppose:

5^8

Instead of:

$5 \times 5 \times 5 \times 5 \times 5 \times 5 \times 5 \times 5$

we can write:

5^8
= $(5^2)^4$
= 25^4
= $(25^2)^2$
= 625^2

So the exponent gets divided by 2.

$8 \rightarrow 4 \rightarrow 2 \rightarrow 1$

This gives approximately:

$O(\log n)$

time.

11. But there is an important issue in your recursive code

Your implementation handles odd powers like this:

```
if(pow % 2 == 1) {
    return (base * modpow(base, pow-1)) % MOD;
}
```

For example:

5^7

becomes:

5×5^6

Then:

5^6
 $\rightarrow 25^3$

So it still works, but this is not the cleanest binary exponentiation implementation.

A better recursive version is:

```

long long modpow(long long base, long long exp) {
    if (exp == 0)
        return 1;

    long long half = modpow(base, exp / 2);
    long long result = (half * half) % MOD;

    if (exp % 2 == 1)
        result = (result * base) % MOD;

    return result;
}

```

This gives a clean:

```
O(log n)
```

recursion for both even and odd exponents.

12. Understanding the recursive version with 5^{13}

Start:

```
modpow(5, 13)
```

Since 13 is odd:

```
modpow(5, 13)
```

calls:

```
modpow(5, 6)
```

Then:

```
modpow(5, 6)
```

becomes:

```
modpow(25, 3)
```

Then:

```
modpow(25, 3)
```

becomes:

```
25 × modpow(25, 2)
```

Then:

```
modpow(25, 2)
```

becomes:

```
modpow(625, 1)
```

Then:

```
625 × modpow(625, 0)
```

Base case:

```
modpow(625, 0) = 1
```

The recursion then returns upward.

The important point is that **even exponents are divided by 2**, which makes it very fast.

13. Why `% MOD` everywhere?

Suppose:

```
ans = ans * base;
```

Both values can become huge.

Instead:

```
ans = (ans * base) % MOD;
```

we keep the number small.

Mathematically:

```
(a × b) % MOD
```

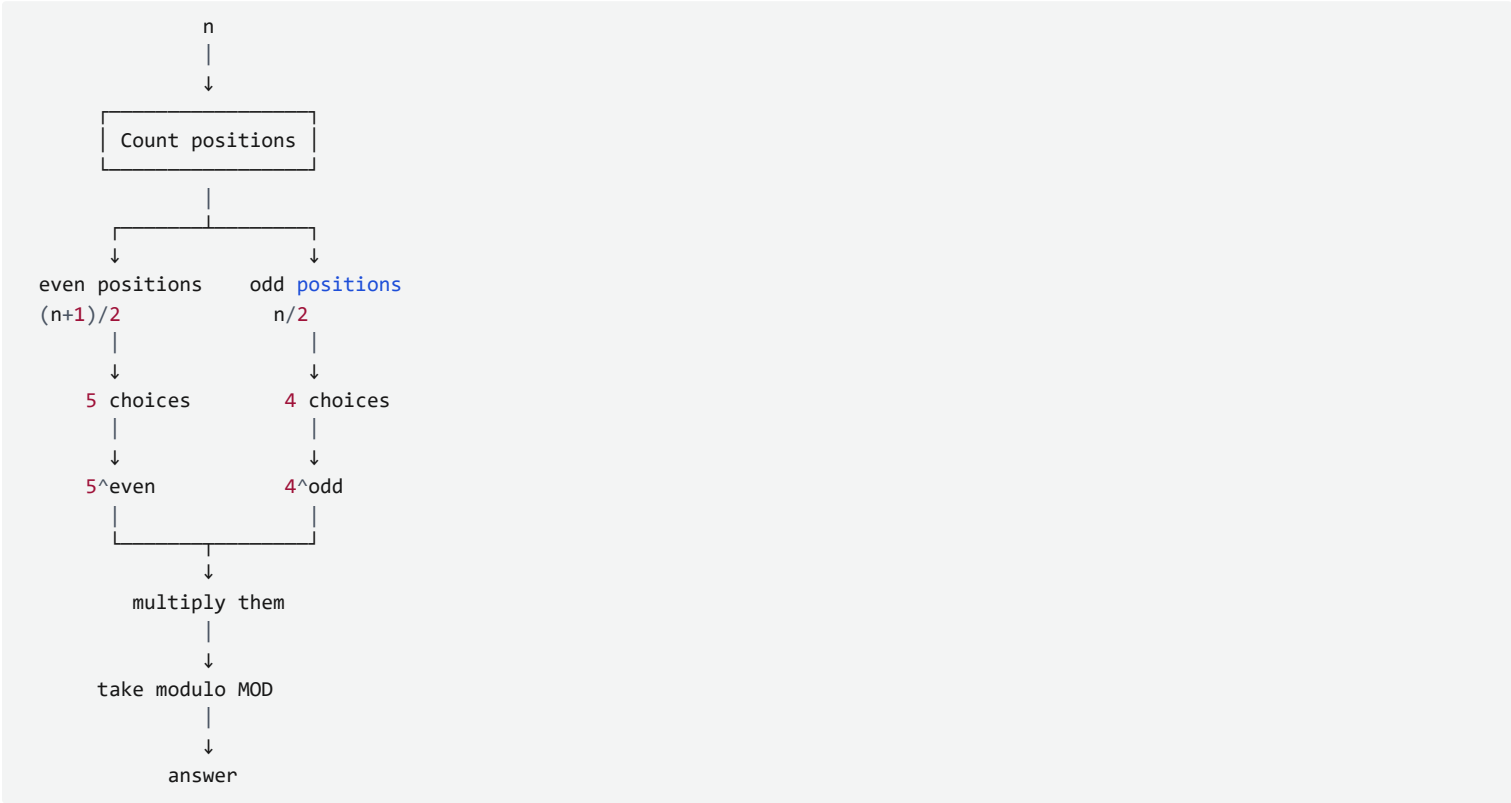
can be calculated using:

```
((a % MOD) × (b % MOD)) % MOD
```

So we can safely keep reducing values.

14. Complete flow of your program

Your program follows this pipeline:



15. Example with n = 6

Positions:

Index:	0	1	2	3	4	5
	E	P	E	P	E	P

Even positions:

```
0, 2, 4
```

So:

```
even = 3
```

Odd positions:

```
1, 3, 5
```

So:

```
odd = 3
```

Therefore:

```
answer = 53 × 43
```

```
= 125 × 64  
= 8000
```

16. Why brute force is impossible

You might initially think:

```
"I'll generate every possible n-digit number and check whether it's good."
```

But each position has up to 10 possible digits.

So there can be roughly:

```
10n
```

numbers.

For:

```
n = 20
```

that's already:

```
1020
```

possibilities.

Impossible.

The clever observation is:

```
We don't need to generate the numbers. We only need to count the choices at each position.
```

That's the main trick.

17. Complexity

There are two important parts.

Counting positions

```
even = (n+1)/2;  
odd = n/2;
```

takes:

```
O(1)
```

Modular exponentiation

```
modpow(5, even)  
modpow(4, odd)
```

takes:

```
O(log n)
```

Therefore overall:

```
Time: O(log n)  
Space: O(log n)
```

for the recursive implementation because of the recursion stack.

If you use your iterative `modpow1()`, then:

```
Time: O(log n)  
Space: O(1)
```

18. One small correction to your comments

You wrote:

```
// Count of even positions (0,2,4,...) in a number of length n
```

That's correct.

But you wrote:

```
// Count of odd positions (1,3,5,...) in a number of length n
```

Also correct.

However, remember that **the problem is talking about indices**, not the numerical value of the digit.

For example:

```
number = 23457

index:  0 1 2 3 4
digit:  2 3 4 5 7
      ↑  ↑  ↑
      even positions
```

The condition is:

```
index even → digit must be even
index odd  → digit must be prime
```

Not:

```
even digit → even index
prime digit → odd index
```

The entire solution in one formula

For a number of length n :

```
number of even positions = ceil(n/2) = (n+1)/2
number of odd positions  = floor(n/2) = n/2
```

Each even position has:

5 choices

Each odd position has:

4 choices

Therefore:

```
Answer = 5(n+1)/2 × 4n/2
```

or in C++:

```
long long even = (n + 1) / 2;
long long odd  = n / 2;

return modpow(5, even) * modpow(4, odd) % MOD;
```

The core pattern to remember for DSA: whenever every position has a fixed number of independent choices, look for a **product of powers** rather than generating all possibilities.